Advanced Day 3 — Session 3 (Hours 9–12)

Python Programming (Advanced) • Project Packages, Logging, Safer File Ops, and Decorators

Session 3 Overview

Topics Covered Today

- Hour 9 (Hour 9 overall): Project structure, packages, imports, and config
- Hour 10 (Hour 10 overall): Logging and error reporting (practical)
- Hour 11 (Hour 11 overall): Context managers and safer file operations
- Hour 12 (Hour 12 overall): Decorators — timing, toy authorization, lightweight validation

Session 3 Outcomes

- Organize the checkpoint core into `src/tracker/` with clean relative imports
- Add practical `logging` that separates developer diagnostics from user messages
- Save JSON safely using write-temp-then-replace instead of direct overwrite
- Write and apply small `@timed` decorators without changing core function behavior

Scope Guardrails for Today

In Scope

- Package layout under `src/tracker/` with `__init__.py` and relative imports
- File-based logging to `logs/app.log`
- Context managers and a `save_json_safe` helper
- Small readable decorators for timing and toy authorization

Not Yet

- Publishing or installing packages (`pyproject.toml`, editable installs, `pip install -e .`)
- Production secrets management or security frameworks
- Decorator factories with arguments, class decorators, or metaclasses

Hour 9: Project Structure, Packages, Imports, and Config

Learning Outcomes

- Move the checkpoint core from a flat script pile into `src/tracker/`
- Use relative imports inside the package (`from .models import Task`)
- Run the demo with `python -m src.tracker.demo` from the project root
- Centralize path constants in a light `config.py`

Why Structure Matters Now

```
Before (flat):                  After (organized):
project_root/                   project_root/
    models.py                       src/
    services.py                         tracker/
    exceptions.py                           __init__.py
    demo.py                                 exceptions.py
                                            models.py
                                            services.py
                                            config.py
                                            demo.py
```

- Upcoming layers need to find the core reliably: logging, persistence, GUI, API, tests
- Structure tells future readers where responsibilities live
- Brittle or accidental imports break when any layer moves

Source

Key Vocabulary

- **Module** — a single `.py` file: `models.py`, `services.py`
- **Package** — a directory containing `__init__.py`: `tracker/`
- **`__init__.py`** — marks a folder as a package; can stay empty
- **Relative import** — `from .models import Task` (dot = current package)
- **`-m` flag** — run a module by import path, preserving package context
- **Entry point** — `src/tracker/demo.py`, run via `python -m src.tracker.demo`

Source

`Advanced/lessons/lecture/Day3_Hour1_Advanced.md` → `## Outcomes, Setup, and Vocabulary`

Relative Imports in `models.py`

```python
# src/tracker/models.py
from .exceptions import ValidationError   # dot = "from this package"


class Task:
    VALID_PRIORITIES = {"low", "medium", "high"}

    def __init__(self, task_id: int, title: str, priority: str = "medium") -> None:
        cleaned = title.strip()
        if not cleaned:
            raise ValidationError("Task title cannot be empty.")
        self.task_id   = task_id
        self.title     = cleaned
        self.priority  = priority.strip().lower()
        self.completed = False

    def to_dict(self) -> dict[str, object]:
        return {"task_id": self.task_id, "title": self.title,
                "priority": self.priority, "completed": self.completed}
```

Dependency Direction and Running Correctly

```
Correct dependency direction:
  demo.py  -->  services.py  -->  models.py
      \               \                \
       \-----------+   \-----------+   \--> exceptions.py
```

- `models.py` must NOT import `services.py` — circular imports cause mysterious errors

```python
# CORRECT — from project root
python -m src.tracker.demo

# BREAKS relative imports — avoid
python src/tracker/demo.py
```

- `-m` preserves the package context so `from .services import TaskService` resolves

Light Config Module

```python
# src/tracker/config.py
import os
from pathlib import Path

# parents[2]: config.py -> tracker/ -> src/ -> project_root
BASE_DIR = Path(__file__).resolve().parents[2]

DATA_DIR = Path(os.environ.get("TRACKER_DATA_DIR", "data"))
DB_PATH  = DATA_DIR / "tracker.db"
LOG_DIR  = BASE_DIR / "logs"
LOG_FILE = LOG_DIR  / "app.log"
```

- One home for constants — no hard-coded paths scattered across modules
- `os.environ.get(...)` allows overrides without a full config framework
- PowerShell override: `$env:TRACKER_DATA_DIR = "data"; python -m src.tracker.demo`

Lab: Restructure Your Tracker

Time: 25 minutes

Required Steps

1. `mkdir src\tracker` and create an empty `src\tracker\__init__.py`
2. Move `models.py`, `services.py`, `exceptions.py`, `demo.py` into `src/tracker/`
3. Update all internal imports to use the dot-relative form
4. Add `config.py` with `DATA_DIR`, `DB_PATH`, `LOG_DIR`, `LOG_FILE`
5. Run `python -m src.tracker.demo` from the project root
6. Confirm happy path and at least one caught `NotFoundError` or `ValidationError`

Source

`Advanced/lessons/lecture/Day3_Hour1_Advanced.md` → `## Guided Lab: Restructure Your Tracker`

Completion Criteria (Hour 9)

- `src/tracker/__init__.py` exists (can be empty)
- Internal imports use the dot-relative form — `from .models import Task`
- No module named after a stdlib module (`logging.py`, `json.py`, etc.)
- `python -m src.tracker.demo` from the project root runs without import errors
- `config.py` holds `DATA_DIR` and `DB_PATH`

Common Pitfalls (Hour 9)

- Running `python src/tracker/demo.py` — package context is missing
- Keeping flat `from models import Task` after moving files into the package
- Circular import: `models.py` importing from `services.py`
- Naming a local file `logging.py` or `json.py` — shadows the standard library
- Skipping `__init__.py` — Python cannot recognize the directory as a package

Hour 10: Logging and Error Reporting (Practical)

Learning Outcomes

- Explain why `logging` replaces scattered `print()` debugging
- Choose the right level: DEBUG, INFO, WARNING, ERROR
- Configure file logging to `logs/app.log` near program startup
- Separate developer diagnostics from friendly user-facing messages

The Central Rule

Logs are for developers. Error messages are for users.

```
Developer view (logs/app.log):
  2026-01-14 10:03:22 WARNING src.tracker.services
    Missing task lookup task_id=999

User view (terminal):
  "Task 999 was not found. Please check the ID and try again."
```

- `print()` mixes audiences, carries no timestamp, no severity, disappears on exit
- `logging` routes information to the right audience and persists it to a file
- Exceptions still control flow — logging records what happened

Source

`Advanced/lessons/lecture/Day3_Hour2_Advanced.md` → `## Opening Bridge` and `## Logging vs print()`

Practical Log Levels

```
DEBUG   — detailed developer-only info; filtered out in production
INFO    — normal important events (service started, task added)
WARNING — unexpected but recoverable (lookup for a missing task)
ERROR   — an operation failed (validation rejected, file not writable)
```

```
logger.debug("Inspecting candidate task_id=%s", task_id)
logger.info("Added task_id=%s title=%r", task.task_id, task.title)
logger.warning("Missing task lookup task_id=%s", task_id)
logger.error("Validation rejected task creation")
```

Source

`Advanced/lessons/lecture/Day3_Hour2_Advanced.md` → `### Practical logging levels`

Logging Config: `config.py` and `demo.py`

```
# src/tracker/config.py  — add log paths
LOG_DIR  = BASE_DIR / "logs"
LOG_FILE = LOG_DIR  / "app.log"
```

```
# src/tracker/demo.py — configure once near startup
import logging
from .config import LOG_DIR, LOG_FILE

def configure_logging() -> None:
    LOG_DIR.mkdir(parents=True, exist_ok=True)   # create logs/ if missing
    logging.basicConfig(
        filename=LOG_FILE,
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s %(message)s",
    )
```

- Must call `LOG_DIR.mkdir(...)` **before** logging fires — the module will not create the directory

Module-Level Logger in `services.py`

```python
# src/tracker/services.py
import logging
from .exceptions import NotFoundError, ValidationError
from .models import Task

logger = logging.getLogger(__name__)   # logger name = "src.tracker.services"

class TaskService:
    def __init__(self) -> None:
        self._tasks: dict[int, Task] = {}
        logger.info("TaskService initialized")

    def add(self, task: Task) -> None:
        if task.task_id in self._tasks:
            logger.warning("Rejected duplicate task_id=%s", task.task_id)
            raise ValidationError(f"Task {task.task_id} already exists.")
        self._tasks[task.task_id] = task
        logger.info("Added task_id=%s title=%r", task.task_id, task.title)

    def get(self, task_id: int) -> Task:
        try:
            return self._tasks[task_id]
        except KeyError as exc:
            logger.warning("Missing task lookup task_id=%s", task_id)
            raise NotFoundError(f"Task {task_id} was not found.") from exc
```

Friendly Messages in `demo.py`

```python
def main() -> None:
    configure_logging()
    service = TaskService()

    try:
        service.get(999)
    except NotFoundError as exc:
        logger.warning("Demo: missing task — %s", exc)
        print("That task was not found. Please check the task ID.")

    try:
        service.add(Task(task_id=1, title="Duplicate"))
    except ValidationError:
        logger.exception("Demo: validation failed on add")   # traceback in log
        print("That task could not be saved. Please check the task details.")
```

- `logger.exception(...)` stores the full traceback in `logs/app.log` — never on screen

Lab: Add Practical Logging

Time: 25 minutes

Required Steps

1. Add `LOG_DIR` and `LOG_FILE` to `config.py`
2. Call `configure_logging()` at startup before creating services
3. Add `logger = logging.getLogger(__name__)` to `services.py`
4. Add at least 3 service-layer log calls (init, add, missing lookup)
5. Trigger one expected error and catch it with a friendly user message
6. Open `logs/app.log` and confirm readable, useful entries are present

Source

`Advanced/lessons/lecture/Day3_Hour2_Advanced.md` → `## Guided Lab: Add Practical Logging`

Completion Criteria (Hour 10)

- `logs/app.log` is created automatically at startup
- At least three useful service-layer log entries appear
- User-facing output in the terminal stays friendly and short
- At least one `logger.exception(...)` stores a traceback in the file (not on screen)

Common Pitfalls (Hour 10)

- Forgetting `LOG_DIR.mkdir(parents=True, exist_ok=True)` — logging silently produces no file
- Calling `logging.basicConfig(...)` after the first log fires — it becomes a no-op
- Logging inside a tight loop — produces a log file too noisy to read
- Printing raw tracebacks to end users instead of routing them to the log
- Naming a project file `logging.py` — shadows the standard library module

Hour 11: Context Managers and Safer File Operations

Learning Outcomes

- Explain what a context manager guarantees beyond "closes the file"
- Use `with` consistently for files and other cleanup-requiring resources
- Explain why opening the real file in write mode can corrupt it mid-write
- Implement `save_json_safe(path, data)` using write-temp-then-replace

Context Manager Mental Model

```
# Manual — cleanup depends on reaching file.close()
file = open("data/tasks.json", "w", encoding="utf-8")
file.write("example")
file.close()    # skipped if exception occurs above

# With context manager — cleanup guaranteed
with Path("data/tasks.json").open("w", encoding="utf-8") as file:
    file.write("example")
# file is closed here, even if an exception was raised
```

- `with` = "enter this context, do work, always run exit cleanup"
- Same pattern works for locks, DB connections, temp directories, network sessions
- `with` guarantees cleanup — it does not guarantee a safe write strategy

Source

`Advanced/lessons/lecture/Day3_Hour3_Advanced.md` → `## Concept Briefing` → `###`

Why Direct Overwrites Can Corrupt Data

```
# "w" mode truncates the real file IMMEDIATELY on open
with Path("data/tasks.json").open("w", encoding="utf-8") as file:
    file.write("[\n")
    file.write('  {"task_id": 2, "title": "Half-written"')
    raise RuntimeError("crash mid-write")   # original content is already gone
# with closed the file — but it closed a damaged file
```

```
Direct overwrite:
  open real file -> old content gone -> write -> maybe fail -> corrupted file

Temp-then-replace:
  write side file -> old file untouched -> replace only after success
```

Source

`Advanced/lessons/lecture/Day3_Hour3_Advanced.md` → `## Concept Briefing` → `### Part 2 and Part 3`

`save_json_safe` — Full Implementation

```python
import json
from pathlib import Path
from typing import Any


def save_json_safe(
    path: str | Path,
    data: Any,
    *,
    simulate_failure: bool = False,
) -> None:
    target_path = Path(path)
    target_path.parent.mkdir(parents=True, exist_ok=True)
    # Same-directory temp works for "tasks.json" AND suffix-less "tasks"
    tmp_path = target_path.with_name(f"{target_path.name}.tmp")

    try:
        with tmp_path.open("w", encoding="utf-8") as file:
            json.dump(data, file, indent=2)
            file.write("\n")              # trailing newline for readability
        if simulate_failure:
            raise RuntimeError("simulated failure before replace")
        tmp_path.replace(target_path)    # only touches target after success
    except Exception:
        tmp_path.unlink(missing_ok=True) # clean up temp, then re-raise
        raise
```

Proving the Pattern Works

```
# Save known-good data
save_json_safe("data/tasks.json", [{"task_id": 1, "title": "Original"}])
before = Path("data/tasks.json").read_text(encoding="utf-8")

# Simulate a mid-write crash
try:
    save_json_safe(
        "data/tasks.json",
        [{"task_id": 9, "title": "Should NOT replace original"}],
        simulate_failure=True,
    )
except RuntimeError as err:
    print(f"Caught: {err}")

after = Path("data/tasks.json").read_text(encoding="utf-8")
print(before == after)                          # True  — original intact
print(Path("data/tasks.json.tmp").exists())     # False — temp cleaned up
```

Source

Lab: Safe Save Utility for Tracker Data

Time: 25 minutes

Required Steps

1. Implement `save_json_safe(path, data)` in `src/tracker/storage.py`
2. Save tracker-style data to `data/tasks.json` (use `task.to_dict()` for objects)
3. Inspect the file with `Get-Content data/tasks.json`
4. Run a simulated failure — confirm original file is unchanged and `.tmp` is gone
5. Explain: "What is one advantage of writing to a temp file first?"

Stretch Options

- Add a `load_json(path)` helper that returns `[]` if the file does not exist
- Log successful saves and failed saves using the Hour 10 logging setup

Completion Criteria and Quick Check (Hour 11)

- `save_json_safe` creates the parent directory automatically
- Temp file is in the same directory as the target
- JSON write happens inside a `with` block
- `tmp_path.replace(target_path)` runs only after the write succeeds
- Simulated failure leaves the original file content unchanged

Question: `with` closes the file reliably. Why is that still not enough to make a direct overwrite safe?

Common Pitfalls (Hour 11)

- Using `.with_suffix(".tmp")` — breaks for files with no extension; use `with_name(f"{name}.tmp")` instead
- Writing the temp file in a different directory — complicates `replace` and may cause cross-device errors
- Skipping `parent.mkdir(parents=True, exist_ok=True)` — `FileNotFoundError` on first run
- Not calling `unlink(missing_ok=True)` in the exception path — leaves stale `.tmp` files
- On Windows: `PermissionError` from `replace()` if the target is locked by an editor, sync tool, or antivirus

Hour 12: Decorators for Timing, Toy Authorization, and Lightweight Validation

Learning Outcomes

- Explain a decorator: a function that takes a function, wraps it, and returns a new callable
- Write `wrapper(*args, **kwargs)` that handles both functions and methods
- Build a `@timed` decorator that logs via the Hour 10 logging setup
- Use `functools.wraps` to preserve function name and docstring

The Problem Decorators Solve

```python
# Duplicated timing code — error-prone to maintain
def add_task(self, title: str) -> int:
    start = time.perf_counter()
    result = self._do_add(title)
    logger.info("add_task completed in %.6f s", time.perf_counter() - start)
    return result

def list_titles(self) -> list[str]:
    start = time.perf_counter()
    result = list(self._titles)
    logger.info("list_titles completed in %.6f s", time.perf_counter() - start)
    return result
```

- Same wrapping code copied into every service method
- Change the format string once — must edit every copy
- Forget `return result` in one copy — silent `None` bug

Decorator Anatomy

```
def my_decorator(func):          # 1. accept the original function
    def wrapper(*args, **kwargs): # 2. inner function collects ALL args
        result = func(*args, **kwargs)  # 3. call original
        return result            # 4. MUST return original result
    return wrapper               # 5. return the wrapper, not a call to it
```

```
@my_decorator
def greet(name: str) -> str:
    return f"Hello, {name}!"

# Equivalent to: greet = my_decorator(greet)
```

- `*args, **kwargs` forwards everything including `self` for methods
- Without `return result`: decorated function silently returns `None`

Source

Professional `@timed` Decorator

```python
import logging
import time
from functools import wraps
from typing import Any, Callable


def timed(func: Callable[..., Any]) -> Callable[..., Any]:
    @wraps(func)   # preserves __name__, __doc__ — use in professional code
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start  = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        logging.getLogger(__name__).info(
            "%s completed in %.6f seconds",
            func.__name__,
            elapsed,
        )
        return result  # original result unchanged
    return wrapper
```

Applying `@timed` to Service Methods

```python
class TaskService:
    def __init__(self) -> None:
        self._titles: list[str] = []

    @timed
    def add_task(self, title: str) -> int:
        self._titles.append(title)
        return len(self._titles)          # still returns correct count

    @timed
    def list_titles(self) -> list[str]:
        return list(self._titles)         # still returns the list
```

```
# logs/app.log after running the demo
INFO src.tracker.services add_task completed in 0.000080 seconds
INFO src.tracker.services list_titles completed in 0.000030 seconds
```

- `self` passes through `args` — no special method support needed

Toy `@requires_api_key` — Shape, Not Security

```python
from functools import wraps
from typing import Any, Callable


def requires_api_key(func: Callable[..., Any]) -> Callable[..., Any]:
    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        if kwargs.get("api_key") != "demo-key":
            raise PermissionError("invalid API key")
        return func(*args, **kwargs)
    return wrapper


@requires_api_key
def fetch_remote_task_titles(*, api_key: str) -> list[str]:
    return ["Plan API client", "Document JSON contract"]
```

- Keyword-only `api_key` avoids positional ambiguity

Readability Rules for Decorators

```
Use a decorator when:
  - the extra behavior repeats across multiple functions
  - the wrapper stays small and focused
  - the decorated function becomes easier to read

Avoid a decorator when:
  - it hides the main business rule
  - it creates surprising side effects
  - debugging becomes harder than the duplication was
```

- Timing is a good decorator: it is cross-cutting, small, and observable
- Validation that IS the business rule belongs in the model or service — not a wrapper

Source

`Advanced/lessons/lecture/Day3_Hour4_Advanced.md` → `## Concept Briefing` → `###`

Lab: Timing Decorator in the Tracker Project

Time: 26 minutes

Required Steps

1. Implement `@timed` using `perf_counter` + `logging.getLogger(__name__).info`
2. Apply `@timed` to at least two service functions (e.g., `add_task`, `list_titles`)
3. Run the demo from the project root
4. Confirm timing messages appear in `logs/app.log`
5. Verify the decorated functions still return the same values as before

Source

`Advanced/lessons/lecture/Day3_Hour4_Advanced.md` → `## Guided Lab: Timing Decorator in the Tracker Project`

Completion Criteria and Pitfalls (Hour 12)

- `@timed` uses `wrapper(*args, **kwargs)` — no fixed parameter list
- `time.perf_counter()` measures elapsed time
- Timing message goes through `logging.getLogger(__name__).info(...)` — not `print()`
- `return result` is present — decorated functions return unchanged values
- `@wraps(func)` preserves `func.__name__` and docstring

Pitfalls:

- Forgetting `return result` — decorated function silently returns `None`
- Not forwarding `*args, **kwargs` — method call fails with missing `self`
- Using `print()` instead of `logging` — timing output is inconsistent and untraceable

Session 3 Wrap-Up

What We Built Today

- A professional package shape under `src/tracker/` that other layers can import reliably
- Practical logging that routes developer detail to `logs/app.log` and friendly messages to users
- A `save_json_safe` helper that protects good data during file writes
- A `@timed` decorator that eliminates repeated timing boilerplate around service methods

Key Day 3 Rule

Operational habits — structure, logging, safe saves, reusable wrappers — are part of the design, not an afterthought.

Day 3 Homework / Study Checklist

- Run `python -m src.tracker.demo` from the project root and confirm it succeeds
- Open `logs/app.log` and explain every entry — what level, what module, what event
- Trigger a simulated save failure and confirm the original file is unchanged
- Confirm a decorated service call still returns the correct value after adding `@timed`
- Check: does `models.py` import anything from `services.py`? It should not

Source Alignment

- `Advanced/assignments/Advanced_Day3_homework.ipynb` → `## Reflection and Submission Checklist`

Next Session Preview

Session 4 (Hours 13–16)

- HTTP clients with `requests` and JSON API contracts
- Environment-variable and secrets habits
- Capstone planning workshop
- Checkpoint 2: persistence-ready core + JSON save/load

Bridge from Today

The same discipline applies to HTTP wrappers: keep wrappers small, use logging for visibility, and make the core service logic easy to read and test.

Thank You!

Package it cleanly.

Log what matters.

Save data safely.

Keep the wrappers readable.